

```

/* =====
Finance Pro — основная логика приложения
Хранит все данные в localStorage браузера/Telegram WebView.
===== */

(function () {
    'use strict';

    var STORAGE_KEY = 'financeProData';

    var DEFAULT_STATE = {
        transactions: [],
        sidejobs: [],
        categories: {
            expense: ['Продукты', 'Транспорт', 'Жильё', 'Развлечения', 'Здоровье', 'Оде
            income: ['Зарплата', 'Подработка', 'Инвестиции', 'Подарки', 'Другое']
        },
        settings: { currency: 'RUB' }
    };

    var state = loadState();

    /* ----- */
    /* Хранилище */
    /* ----- */

    function loadState() {
        try {
            var raw = localStorage.getItem(STORAGE_KEY);
            if (!raw) return deepClone(DEFAULT_STATE);
            var parsed = JSON.parse(raw);
            // подстраховка на случай отсутствия каких-то полей в старых сохранениях
            return Object.assign(deepClone(DEFAULT_STATE), parsed, {
                categories: Object.assign(deepClone(DEFAULT_STATE.categories), parsed.cat
                settings: Object.assign(deepClone(DEFAULT_STATE.settings), parsed.setting
            });
        } catch (e) {
            console.error('Не удалось прочитать сохранённые данные', e);
            return deepClone(DEFAULT_STATE);
        }
    }

    function saveState() {
        try {

```

```

        localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
    } catch (e) {
        console.error('Не удалось сохранить данные', e);
        toast('Не удалось сохранить данные на устройстве', 'error');
    }
}

function deepClone(obj) {
    return JSON.parse(JSON.stringify(obj));
}

function uid() {
    return Date.now().toString(36) + Math.random().toString(36).slice(2, 8);
}

/* ----- */
/* Форматирование */
/* ----- */

var CURRENCY_SYMBOLS = { RUB: '₽', USD: '$', EUR: '€', CRYPTO: '฿' };

function formatMoney(amount, currency) {
    var symbol = CURRENCY_SYMBOLS[currency || state.settings.currency] || '₽';
    var value = Number(amount) || 0;
    var formatted = value.toLocaleString('ru-RU', { minimumFractionDigits: 2, max
    return formatted + ' ' + symbol;
}

function formatDate(dateStr) {
    if (!dateStr) return '-';
    var d = new Date(dateStr);
    if (isNaN(d.getTime())) return dateStr;
    return d.toLocaleDateString('ru-RU', { day: '2-digit', month: '2-digit', year
}

function todayISO() {
    var d = new Date();
    var offset = d.getTimezoneOffset();
    d = new Date(d.getTime() - offset * 60000);
    return d.toISOString().slice(0, 10);
}

function isSameMonth(dateStr, ref) {
    var d = new Date(dateStr);
    return d.getFullYear() === ref.getFullYear() && d.getMonth() === ref.getMonth
}

```

```

/* ----- */
/* Toasts */
/* ----- */

function toast(message, kind) {
    var container = document.getElementById('toastContainer');
    if (!container) return;
    var el = document.createElement('div');
    el.className = 'toast' + (kind === 'error' ? ' toast--error' : '');
    el.textContent = message;
    container.appendChild(el);
    setTimeout(function () {
        el.remove();
    }, 3200);
}

/* ----- */
/* Модальные окна */
/* ----- */

function openModal(id) {
    var overlay = document.getElementById('modalOverlay');
    var modal = document.getElementById(id);
    if (!overlay || !modal) return;
    overlay.removeAttribute('hidden');
    modal.removeAttribute('hidden');
}

function closeAllModals() {
    var overlay = document.getElementById('modalOverlay');
    if (overlay) overlay.setAttribute('hidden', '');
    document.querySelectorAll('.modal').forEach(function (m) { m.setAttribute('hi
}

var confirmCallback = null;
function showConfirm(text, onConfirm, title) {
    document.getElementById('modalConfirmTitle').textContent = title || 'Подтверд
    document.getElementById('modalConfirmText').textContent = text;
    confirmCallback = onConfirm;
    openModal('modalConfirm');
}

/* ----- */
/* Боковое меню (открытие/закрытие) */
/* ----- */

function initSidebar() {

```

```

var menuToggle = document.getElementById('menuToggle');
var sidebar = document.getElementById('sidebar');
var sidebarOverlay = document.getElementById('sidebarOverlay');
var sidebarClose = document.getElementById('sidebarClose');

function openSidebar() {
    if (!sidebar) return;
    sidebar.classList.add('sidebar--open');
    if (sidebarOverlay) sidebarOverlay.classList.add('sidebar-overlay--visible')
}

function closeSidebar() {
    if (!sidebar) return;
    sidebar.classList.remove('sidebar--open');
    if (sidebarOverlay) sidebarOverlay.classList.remove('sidebar-overlay--visib
}

if (menuToggle && sidebar) {
    menuToggle.addEventListener('click', function () {
        sidebar.classList.contains('sidebar--open') ? closeSidebar() : openSideba
    });
}

if (sidebarOverlay) sidebarOverlay.addEventListener('click', closeSidebar);
if (sidebarClose) sidebarClose.addEventListener('click', closeSidebar);
document.addEventListener('keydown', function (e) { if (e.key === 'Escape') c
window.addEventListener('resize', function () { if (window.innerWidth > 960)

window.__closeSidebar = closeSidebar;
}

/* ----- */
/* Навигация между страницами */
/* ----- */

function initNavigation() {
    var titles = {
        dashboard: 'Главная',
        transactions: 'Доходы и расходы',
        sidejobs: 'Подработки',
        analytics: 'Аналитика',
        calendar: 'Календарь',
        reports: 'Отчеты',
        settings: 'Настройки'
    };
    var pageTitle = document.getElementById('pageTitle');

    function goToPage(pageId) {
        if (!document.getElementById('page-' + pageId)) return;

```

```

document.querySelectorAll('.page').forEach(function (p) { p.classList.remove
document.getElementById('page-' + pageId).classList.add('active');
document.querySelectorAll('[data-page]').forEach(function (l) {
    l.classList.toggle('active', l.getAttribute('data-page') === pageId);
});
if (pageTitle && titles[pageId]) pageTitle.textContent = titles[pageId];
if (window.__closeSidebar) window.__closeSidebar();

// Страница только что стала видимой — её канвасы могли быть
// отрисованы ещё при display:none (ширина 0). Перерисовываем.
if (window.requestAnimationFrame) {
    requestAnimationFrame(function () { renderVisibleCharts(); });
} else {
    renderVisibleCharts();
}
}

document.querySelectorAll('[data-page]').forEach(function (link) {
    link.addEventListener('click', function (e) {
        var pageId = link.getAttribute('data-page');
        if (!pageId) return;
        e.preventDefault();
        goToPage(pageId);
    });
});

document.querySelectorAll('[data-page-link]').forEach(function (link) {
    link.addEventListener('click', function (e) {
        e.preventDefault();
        goToPage(link.getAttribute('data-page-link'));
    });
});
}

/* ----- */
/* Категории */
/* ----- */

function allCategories() {
    return state.categories.expense.concat(state.categories.income);
}

function populateTransactionCategorySelect(type) {
    var select = document.getElementById('transactionCategory');
    if (!select) return;
    var list = state.categories[type] || [];
    select.innerHTML = list.map(function (c) { return '<option value="' + escapeH

```

```

}

function populateFilterCategorySelect() {
  var select = document.getElementById('filterCategory');
  if (!select) return;
  var current = select.value;
  var unique = Array.from(new Set(allCategories()));
  select.innerHTML = '<option value="all">Все категории</option>' +
    unique.map(function (c) { return '<option value="' + escapeHtml(c) + '">' +
    select.value = current || 'all';
}

function renderCategoriesList() {
  var list = document.getElementById('categoriesList');
  if (!list) return;
  var rows = [];
  ['expense', 'income'].forEach(function (type) {
    state.categories[type].forEach(function (name) {
      rows.push(
        '<li class="list__item" data-type="' + type + '" data-name="' + escapeH
        '<span>' + escapeHtml(name) + ' <span class="card__hint">(' + (type ===
        '<button type="button" class="icon-btn" data-delete-category aria-label
        '</li>'
      );
    });
  });
  list.innerHTML = rows.join('') || '<li class="list__item">Категорий пока нет<

list.querySelectorAll('[data-delete-category]').forEach(function (btn) {
  btn.addEventListener('click', function () {
    var li = btn.closest('li');
    var type = li.getAttribute('data-type');
    var name = li.getAttribute('data-name');
    showConfirm('Удалить категорию «' + name + '»? ', function () {
      state.categories[type] = state.categories[type].filter(function (c) { r
      saveState();
      renderCategoriesList();
      populateFilterCategorySelect();
      populateTransactionCategorySelect(getActiveTransactionType());
      toast('Категория удалена');
    });
  });
});
}

function initCategories() {
  var addBtn = document.getElementById('addCategoryBtn');

```

```

if (addBtn) {
  addBtn.addEventListener('click', function () {
    var nameInput = document.getElementById('newCategoryName');
    var typeSelect = document.getElementById('newCategoryType');
    var name = nameInput.value.trim();
    if (!name) { toast('Введите название категории', 'error'); return; }
    var type = typeSelect.value;
    if (state.categories[type].indexOf(name) !== -1) { toast('Такая категория
state.categories[type].push(name);
    saveState();
    nameInput.value = '';
    renderCategoriesList();
    populateFilterCategorySelect();
    populateTransactionCategorySelect(getActiveTransactionType());
    toast('Категория добавлена');
  });
}
}

/* ----- */
/* Операции (доходы/расходы) */
/* ----- */

function getActiveTransactionType() {
  var active = document.querySelector('#transactionTypeSwitch .segmented-control
return active ? active.getAttribute('data-type') : 'expense';
}

function initTransactionForm() {
  var switchEl = document.getElementById('transactionTypeSwitch');
  if (switchEl) {
    switchEl.querySelectorAll('.segmented-control__item').forEach(function (btn) {
      btn.addEventListener('click', function () {
        switchEl.querySelectorAll('.segmented-control__item').forEach(function (
          btn.classList.add('active');
          populateTransactionCategorySelect(btn.getAttribute('data-type'));
        });
      });
    }
  }

  var form = document.getElementById('formTransaction');
  if (form) {
    form.addEventListener('submit', function (e) {
      e.preventDefault();
      var amount = parseFloat(document.getElementById('transactionAmount').valu
      if (!amount || amount <= 0) { toast('Укажите корректную сумму', 'error');

```

```

var tx = {
  id: uid(),
  type: getActiveTransactionType(),
  amount: amount,
  category: document.getElementById('transactionCategory').value || 'Другой',
  subcategory: document.getElementById('transactionSubcategory').value || 'Другой',
  date: document.getElementById('transactionDate').value || todayISO(),
  time: document.getElementById('transactionTime').value,
  paymentMethod: document.getElementById('transactionPaymentMethod').value || 'Другой',
  comment: document.getElementById('transactionComment').value.trim()
};

state.transactions.push(tx);
saveState();

renderDashboard();
renderTransactionsTable();
populateFilterCategorySelect();
closeAllModals();
form.reset();
document.getElementById('transactionDate').value = todayISO();
toast(tx.type === 'income' ? 'Доход добавлен' : 'Расход добавлен');
hapticFeedback('success');
});
}
}

function deleteTransaction(id) {
  showConfirm('Удалить операцию?', function () {
    state.transactions = state.transactions.filter(function (t) { return t.id !== id; });
    saveState();
    renderDashboard();
    renderTransactionsTable();
    toast('Операция удалена');
  });
}

function getFilteredTransactions() {
  var type = (document.getElementById('filterType') || {}).value || 'all';
  var category = (document.getElementById('filterCategory') || {}).value || 'all';
  var search = ((document.getElementById('filterSearch') || {}).value || '').toLowerCase();
  var from = (document.getElementById('filterDateFrom') || {}).value;
  var to = (document.getElementById('filterDateTo') || {}).value;

  return state.transactions
    .filter(function (t) { return type === 'all' || t.type === type; })
    .filter(function (t) { return category === 'all' || t.category === category

```



```

        .filter(function (t) {
            if (!search) return true;
            return (t.category + ' ' + t.subcategory + ' ' + t.comment).toLowerCase()
        })
        .filter(function (t) { return !from || t.date >= from; })
        .filter(function (t) { return !to || t.date <= to; })
        .sort(function (a, b) { return (b.date + (b.time || '')).localeCompare(a.date + (a.time || '')) });
    }

function renderTransactionsTable() {
    var body = document.getElementById('transactionsBody');
    if (!body) return;
    var list = getFilteredTransactions();

    if (!list.length) {
        body.innerHTML = '<tr class="table__empty-row"><td colspan="7">Операций пока нет</td></tr>';
        return;
    }

    body.innerHTML = list.map(function (t) {
        var amountClass = t.type === 'income' ? 'amount-positive' : 'amount-negative';
        var sign = t.type === 'income' ? '+' : '-';
        return '<tr>' +
            '<td>' + formatDate(t.date) + '</td>' +
            '<td>' + escapeHtml(t.category) + '</td>' +
            '<td>' + escapeHtml(t.subcategory || '-') + '</td>' +
            '<td>' + escapeHtml(t.comment || '-') + '</td>' +
            '<td>' + paymentMethodLabel(t.paymentMethod) + '</td>' +
            '<td class="table__align-right ' + amountClass + '">' + sign + ' ' + formatAmount(t.amount) + '</td>' +
            '<td><button type="button" class="icon-btn" data-delete-tx="' + t.id + '"></td>' +
            '</tr>';
    }).join('');

    body.querySelectorAll('[data-delete-tx]').forEach(function (btn) {
        btn.addEventListener('click', function () { deleteTransaction(btn.getAttribute('data-delete-tx')); });
    });
}

function paymentMethodLabel(v) {
    return { cash: 'Наличные', card: 'Карта', transfer: 'Перевод', other: 'Другое' }[v];
}

function initTransactionFilters() {
    ['filterType', 'filterCategory', 'filterSearch', 'filterDateFrom', 'filterDateTo'].forEach(function (id) {
        var el = document.getElementById(id);
        if (el) el.addEventListener('input', renderTransactionsTable);
    });
}

```

```

}

/* ----- */
/* Дашборд */
/* ----- */

function renderDashboard() {
    var now = new Date();
    var monthTx = state.transactions.filter(function (t) { return isSameMonth(t.d
    var monthTxIncome = sum(monthTx.filter(function (t) { return t.type === 'inco
    var monthExpense = sum(monthTx.filter(function (t) { return t.type === 'expen

    // Доход от подработок (только оплаченные) тоже считается доходом за месяц.
    var monthSidejobsIncome = sum(
        state.sidejobs.filter(function (j) { return j.status === 'paid' && isSameMo
        'amount'
    );
    var monthIncome = monthTxIncome + monthSidejobsIncome;

    var allIncome = sum(state.transactions.filter(function (t) { return t.type ==
    var allExpense = sum(state.transactions.filter(function (t) { return t.type =
    var paidSidejobsEarned = sum(state.sidejobs.filter(function (j) { return j.st

    // «Все деньги, которые есть» – весь чистый доход по операциям + все оплаченн
    // Ничего не считается отдельно – любая новая запись сразу меняет эту сумму.
    var totalBalance = (allIncome - allExpense) + paidSidejobsEarned;

    setText('totalBalance', formatMoney(totalBalance));
    setText('totalIncome', formatMoney(monthIncome));
    setText('totalExpense', formatMoney(monthExpense));

    var incomeDeltaEl = document.getElementById('incomeDelta');
    if (incomeDeltaEl) {
        var monthIncomeOpsCount = monthTx.filter(function (t) { return t.type === '
            + state.sidejobs.filter(function (j) { return j.status === 'paid' && isSa
        incomeDeltaEl.textContent = monthIncomeOpsCount + ' операций за месяц';
    }
    var expenseDeltaEl = document.getElementById('expenseDelta');
    if (expenseDeltaEl) expenseDeltaEl.textContent = monthTx.filter(function (t)

    setText('balanceHint', state.transactions.length ? 'Обновлено сейчас' : 'Мало

    var monthSidejobs = state.sidejobs.filter(function (j) { return isSameMonth(j
    setText('sidejobsMonthEarned', formatMoney(sum(monthSidejobs, 'amount')));
    setText('sidejobsMonthHours', formatHours(sum(monthSidejobs, 'hours')) + ' ч

    // Финансовая подушка: остаток от дохода за месяц

```

```

var pillowPercent = monthIncome > 0 ? Math.max(0, Math.min(100, Math.round(((
setText('pillowPercent', pillowPercent + '%');
var pillowRing = document.getElementById('pillowRing');
if (pillowRing) pillowRing.setAttribute('data-value', pillowPercent);
setText('pillowNote', 'Остаток от дохода · ' + (monthIncome > 0 ? Math.round(
var avgMonthlyExpense = monthExpense || (allExpense / Math.max(1, monthsWithD
var pillowMonths = avgMonthlyExpense > 0 ? (totalBalance / avgMonthlyExpense)
setText('pillowMonths', formatMoney(totalBalance));
setText('pillowMonthsLabel', 'Хватит на ' + Math.max(0, Math.floor(pillowMont

// Здоровье финансов — простая эвристика
var health = 0;
if (state.transactions.length) {
    var savingsRate = monthIncome > 0 ? (monthIncome - monthExpense) / monthInc
    health = Math.max(0, Math.min(100, Math.round(50 + savingsRate * 50)));
}
setText('healthScore', health);
var ring = document.getElementById('healthRing');
if (ring) ring.setAttribute('data-value', health);
setText('healthHint', state.transactions.length ? (health >= 60 ? 'Хорошо' :
setText('healthNote', state.transactions.length
    ? 'Расчёт на основе доходов и расходов за текущий месяц.'
    : 'Внесите первые данные, чтобы рассчитать показатели здоровья.');
```

```

renderRecentTransactions();
renderBalanceMiniChart();
renderIncomeMiniChart();
renderExpenseMiniChart();
renderBalanceHistoryChart();
renderAnalyticsCharts();
}

function formatHours(h) {
    var n = Number(h) || 0;
    return (Math.round(n * 100) / 100).toString().replace('.', ',');
}

function debounce(fn, wait) {
    var t;
    return function () {
        clearTimeout(t);
        var args = arguments;
        t = setTimeout(function () { fn.apply(null, args); }, wait);
    };
}

function monthsWithData() {
```

```

    var months = new Set(state.transactions.map(function (t) { return (t.date ||
    return months.size || 1;
}

function renderRecentTransactions() {
    var body = document.getElementById('recentTransactionsBody');
    if (!body) return;
    var list = state.transactions
        .slice()
        .sort(function (a, b) { return (b.date + (b.time || '')).localeCompare(a.da
        .slice(0, 5);

    if (!list.length) {
        body.innerHTML = '<tr class="table__empty-row"><td colspan="4">Пока нет оне
        return;
    }

    body.innerHTML = list.map(function (t) {
        var amountClass = t.type === 'income' ? 'amount-positive' : 'amount-negativ
        var sign = t.type === 'income' ? '+' : '-';
        return '<tr>' +
            '<td>' + escapeHtml(t.category) + '</td>' +
            '<td>' + escapeHtml(t.comment || '-') + '</td>' +
            '<td>' + formatDate(t.date) + '</td>' +
            '<td class="table__align-right ' + amountClass + '">' + sign + ' ' + form
            '</tr>';
    }).join('');
}

/* ----- */
/* Подработки */
/* ----- */

function computeSidejobHours(timeStart, timeEnd) {
    if (!timeStart || !timeEnd) return 0;
    var startParts = timeStart.split(':').map(Number);
    var endParts = timeEnd.split(':').map(Number);
    var startMinutes = startParts[0] * 60 + startParts[1];
    var endMinutes = endParts[0] * 60 + endParts[1];
    var diff = endMinutes - startMinutes;
    if (diff <= 0) diff += 24 * 60; // работа через полночь
    return Math.round((diff / 60) * 100) / 100;
}

function updateSidejobComputedFields() {
    var timeStart = document.getElementById('sidejobTimeStart').value;
    var timeEnd = document.getElementById('sidejobTimeEnd').value;

```

```

var amount = parseFloat(document.getElementById('sidejobAmount').value) || 0;
var hours = computeSidejobHours(timeStart, timeEnd);
var rate = hours > 0 ? amount / hours : 0;

document.getElementById('sidejobHours').value = hours ? formatHours(hours) +
document.getElementById('sidejobRate').value = (hours > 0 && amount > 0) ? fo
}

function initSidejobForm() {
  var form = document.getElementById('formSidejob');
  if (!form) return;

  ['sidejobTimeStart', 'sidejobTimeEnd', 'sidejobAmount'].forEach(function (id)
    var el = document.getElementById(id);
    if (el) el.addEventListener('input', updateSidejobComputedFields);
  });

  form.addEventListener('submit', function (e) {
    e.preventDefault();
    var timeStart = document.getElementById('sidejobTimeStart').value;
    var timeEnd = document.getElementById('sidejobTimeEnd').value;
    var amount = parseFloat(document.getElementById('sidejobAmount').value) ||
    var hours = computeSidejobHours(timeStart, timeEnd);
    var rate = hours > 0 ? amount / hours : 0;

    var job = {
      id: uid(),
      title: document.getElementById('sidejobTitle').value.trim(),
      client: document.getElementById('sidejobClient').value.trim(),
      company: document.getElementById('sidejobCompany').value.trim(),
      date: document.getElementById('sidejobDate').value || todayISO(),
      timeStart: timeStart,
      timeEnd: timeEnd,
      hours: hours,
      rate: rate,
      amount: amount,
      status: document.getElementById('sidejobStatus').value,
      comment: document.getElementById('sidejobComment').value.trim()
    };

    if (!job.title) { toast('Укажите название работы', 'error'); return; }
    if (!amount || amount <= 0) { toast('Укажите заработанную сумму', 'error');

    state.sidejobs.push(job);
    saveState();
    renderSidejobs();
    renderDashboard();
    closeAllModals();

```

```

        form.reset();
        document.getElementById('sidejobDate').value = todayISO();
        document.getElementById('sidejobHours').value = '';
        document.getElementById('sidejobRate').value = '';
        toast('Подработка добавлена');
        hapticFeedback('success');
    });
}

function renderSidejobs() {
    var body = document.getElementById('sidejobsBody');
    if (!body) return;

    var totalEarned = sum(state.sidejobs, 'amount');
    var totalHours = sum(state.sidejobs, 'hours');
    var avgRate = totalHours > 0 ? totalEarned / totalHours : 0;

    setText('sidejobsTotalEarned', formatMoney(totalEarned));
    setText('sidejobsTotalHours', formatHours(totalHours));
    setText('sidejobsAvgRate', formatMoney(avgRate) + ' /ч');

    var byMonth = {};
    state.sidejobs.forEach(function (j) {
        var key = (j.date || '').slice(0, 7);
        byMonth[key] = (byMonth[key] || 0) + j.amount;
    });
    var bestMonth = Object.keys(byMonth).sort(function (a, b) { return byMonth[b] > byMonth[a]; })[0];
    setText('sidejobsBestMonth', bestMonth ? formatMonthLabel(bestMonth) : '-');

    var statusFilter = (document.getElementById('sidejobsFilterStatus') || {}).value;
    var list = state.sidejobs
        .filter(function (j) { return statusFilter === 'all' || j.status === statusFilter; })
        .sort(function (a, b) { return b.date.localeCompare(a.date); });

    if (!list.length) {
        body.innerHTML = '<tr class="table__empty-row"><td colspan="8">Подработок п
        return;
    }

    body.innerHTML = list.map(function (j) {
        var statusLabel = j.status === 'paid' ? 'Оплачено' : 'Ожидает оплаты';
        return '<tr>' +
            '<td>' + escapeHtml(j.title) + '</td>' +
            '<td>' + escapeHtml(j.client || j.company || '-') + '</td>' +
            '<td>' + formatDate(j.date) + '</td>' +
            '<td class="table__nowrap">' + formatHours(j.hours) + ' ч</td>' +
            '<td class="table__nowrap">' + formatMoney(j.rate) + ' /ч</td>' +

```

```

        '<td class="table__align-right">' + formatMoney(j.amount) + '</td>' +
        '<td>' + statusLabel + '</td>' +
        '<td><button type="button" class="icon-btn" data-delete-job="' + j.id + ' '
        '</tr>';
    })).join('');

body.querySelectorAll('[data-delete-job]').forEach(function (btn) {
    btn.addEventListener('click', function () {
        var id = btn.getAttribute('data-delete-job');
        showConfirm('Удалить запись о подработке?', function () {
            state.sidejobs = state.sidejobs.filter(function (j) { return j.id !== id });
            saveState();
            renderSidejobs();
            renderDashboard();
            toast('Запись удалена');
        });
    });
});
}

function formatMonthLabel(key) {
    var parts = key.split('-');
    var d = new Date(parseInt(parts[0], 10), parseInt(parts[1], 10) - 1, 1);
    return d.toLocaleDateString('ru-RU', { month: 'long', year: 'numeric' });
}

function initSidejobFilters() {
    var el = document.getElementById('sidejobsFilterStatus');
    if (el) el.addEventListener('change', renderSidejobs);
}

/* ----- */
/* Настройки */
/* ----- */

function initSettings() {
    var currencySelect = document.getElementById('settingsCurrency');
    if (currencySelect) {
        currencySelect.value = state.settings.currency;
        currencySelect.addEventListener('change', function () {
            state.settings.currency = currencySelect.value;
            saveState();
            var el = document.getElementById('userCurrency');
            if (el) el.textContent = state.settings.currency + ' ' + CURRENCY_SYMBOLS;
            renderDashboard();
            renderTransactionsTable();
            toast('Валюта изменена');
        });
    }
}

```

```

    });
    var userCurrencyEl = document.getElementById('userCurrency');
    if (userCurrencyEl) userCurrencyEl.textContent = state.settings.currency +
}

var exportBtn = document.getElementById('exportBackupBtn');
if (exportBtn) {
    exportBtn.addEventListener('click', function () {
        var blob = new Blob([JSON.stringify(state, null, 2)], { type: 'applicatio
        var url = URL.createObjectURL(blob);
        var a = document.createElement('a');
        a.href = url;
        a.download = 'finance-pro-backup-' + todayISO() + '.json';
        document.body.appendChild(a);
        a.click();
        a.remove();
        URL.revokeObjectURL(url);
        toast('Резервная копия скачана');
    });
}

var importBtn = document.getElementById('importDataBtn');
var importInput = document.getElementById('importFileInput');
if (importBtn && importInput) {
    importBtn.addEventListener('click', function () { importInput.click(); });
    importInput.addEventListener('change', function () {
        var file = importInput.files[0];
        if (!file) return;
        var reader = new FileReader();
        reader.onload = function () {
            try {
                var parsed = JSON.parse(reader.result);
                state = Object.assign(deepClone(DEFAULT_STATE), parsed, {
                    categories: Object.assign(deepClone(DEFAULT_STATE.categories), pars
                    settings: Object.assign(deepClone(DEFAULT_STATE.settings), parsed.s
                });
                saveState();
                renderAll();
                toast('Данные импортированы');
            } catch (err) {
                toast('Файл повреждён или имеет неверный формат', 'error');
            }
        };
        reader.readAsText(file);
        importInput.value = '';
    });
}

```



```

var clearBtn = document.getElementById('clearAllDataBtn');
if (clearBtn) {
    clearBtn.addEventListener('click', function () {
        showConfirm('Все данные будут удалены безвозвратно. Продолжить?', function () {
            state = deepClone(DEFAULT_STATE);
            saveState();
            renderAll();
            toast('Все данные удалены');
        }, 'Удалить все данные');
    });
}

/* ----- */
/* Графики (canvas) */
/* ----- */

var CHART_COLORS = ['#2A8A67', '#C0A053', '#3E7CB1', '#A6402C', '#7A5CC0', '#3E

function prepCanvas(canvas) {
    if (!canvas) return null;
    var parent = canvas.parentElement;
    var rect = parent.getBoundingClientRect();
    var w = Math.max(40, rect.width);
    var h = Math.max(40, rect.height || 220);
    var dpr = window.devicePixelRatio || 1;
    canvas.width = w * dpr;
    canvas.height = h * dpr;
    canvas.style.width = w + 'px';
    canvas.style.height = h + 'px';
    var ctx = canvas.getContext('2d');
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.clearRect(0, 0, w, h);
    return { ctx: ctx, w: w, h: h };
}

function setChartEmpty(containerId, isEmpty) {
    var container = document.getElementById(containerId);
    if (!container) return;
    container.classList.toggle('chart-container--empty', !!isEmpty);
}

function niceMax(value) {
    if (value <= 0) return 1;
    var pow = Math.pow(10, Math.floor(Math.log10(value)));
    var n = value / pow;

```

```

    var niceN = n <= 1 ? 1 : n <= 2 ? 2 : n <= 5 ? 5 : 10;
    return niceN * pow;
}

function drawSparkline(canvas, values, color) {
    var setup = prepCanvas(canvas);
    if (!setup || values.length < 2) return;
    var ctx = setup.ctx, w = setup.w, h = setup.h;
    var min = Math.min.apply(null, values);
    var max = Math.max.apply(null, values);
    if (min === max) { min -= 1; max += 1; }
    var pad = 4;
    var stepX = (w - pad * 2) / (values.length - 1);

    function pointAt(i) {
        var x = pad + stepX * i;
        var y = h - pad - ((values[i] - min) / (max - min)) * (h - pad * 2);
        return [x, y];
    }

    // область под линией
    var grad = ctx.createLinearGradient(0, 0, 0, h);
    grad.addColorStop(0, color + '33');
    grad.addColorStop(1, color + '00');
    ctx.beginPath();
    ctx.moveTo(pad, h);
    values.forEach(function (v, i) { var p = pointAt(i); ctx.lineTo(p[0], p[1]);
    ctx.lineTo(w - pad, h);
    ctx.closePath();
    ctx.fillStyle = grad;
    ctx.fill();

    // линия
    ctx.beginPath();
    values.forEach(function (v, i) {
        var p = pointAt(i);
        if (i === 0) ctx.moveTo(p[0], p[1]); else ctx.lineTo(p[0], p[1]);
    });
    ctx.strokeStyle = color;
    ctx.lineWidth = 2;
    ctx.lineJoin = 'round';
    ctx.lineCap = 'round';
    ctx.stroke();

    // последняя точка
    var last = pointAt(values.length - 1);
    ctx.beginPath();

```

```

    ctx.arc(last[0], last[1], 3.5, 0, Math.PI * 2);
    ctx.fillStyle = color;
    ctx.fill();
}

function drawLineChart(canvas, labels, series, opts) {
    var setup = prepCanvas(canvas);
    if (!setup) return;
    var ctx = setup.ctx, w = setup.w, h = setup.h;
    opts = opts || {};
    var padL = 46, padR = 14, padT = 16, padB = 26;
    var plotW = w - padL - padR, plotH = h - padT - padB;

    var allValues = [];
    series.forEach(function (s) { allValues = allValues.concat(s.values); });
    var maxAbs = niceMax(Math.max.apply(null, allValues.map(Math.abs).concat([1]))
    var allowNegative = allValues.some(function (v) { return v < 0; });
    var min = allowNegative ? -maxAbs : 0;
    var max = maxAbs;

    // сетка + подписи оси Y
    ctx.strokeStyle = '#E8ECE8';
    ctx.fillStyle = '#98A69E';
    ctx.font = '10.5px Manrope, sans-serif';
    ctx.textBaseline = 'middle';
    var ySteps = 4;
    for (var i = 0; i <= ySteps; i++) {
        var val = min + ((max - min) / ySteps) * i;
        var y = padT + plotH - ((val - min) / (max - min)) * plotH;
        ctx.beginPath();
        ctx.moveTo(padL, y);
        ctx.lineTo(w - padR, y);
        ctx.stroke();
        ctx.textAlign = 'right';
        ctx.fillText(formatCompactNumber(val), padL - 8, y);
    }

    // ось X подписи
    ctx.textAlign = 'center';
    ctx.textBaseline = 'top';
    var stepX = labels.length > 1 ? plotW / (labels.length - 1) : 0;
    labels.forEach(function (label, i) {
        var x = padL + stepX * i;
        ctx.fillText(label, x, h - padB + 8);
    });

    // линии серий

```

```

series.forEach(function (s, si) {
    var color = s.color || CHART_COLORS[si % CHART_COLORS.length];
    var pts = s.values.map(function (v, i) {
        return [padL + stepX * i, padT + plotH - ((v - min) / (max - min)) * plotH];
    });

    // область под графиком — мягкая заливка градиентом
    if (s.fill && pts.length > 1) {
        var areaGrad = ctx.createLinearGradient(0, padT, 0, padT + plotH);
        areaGrad.addColorStop(0, color + '2E');
        areaGrad.addColorStop(1, color + '00');
        ctx.beginPath();
        ctx.moveTo(pts[0][0], padT + plotH);
        ctx.lineTo(pts[0][0], pts[0][1]);
        tracePath(ctx, pts);
        ctx.lineTo(pts[pts.length - 1][0], padT + plotH);
        ctx.closePath();
        ctx.fillStyle = areaGrad;
        ctx.fill();
    }

    ctx.beginPath();
    tracePath(ctx, pts);
    ctx.strokeStyle = color;
    ctx.lineWidth = 2.5;
    ctx.lineJoin = 'round';
    ctx.lineCap = 'round';
    if (s.dashed) ctx.setLineDash([5, 4]); else ctx.setLineDash([]);
    ctx.stroke();
    ctx.setLineDash([]);

    var dotRadius = pts.length > 40 ? 0 : 3;
    if (dotRadius) {
        pts.forEach(function (p) {
            ctx.beginPath();
            ctx.arc(p[0], p[1], dotRadius, 0, Math.PI * 2);
            ctx.fillStyle = '#fff';
            ctx.fill();
            ctx.lineWidth = 2;
            ctx.strokeStyle = color;
            ctx.stroke();
        });
    }
});

// нулевая линия, если есть отрицательные значения
if (allowNegative) {

```

```

    var zeroY = padT + plotH - ((0 - min) / (max - min)) * plotH;
    ctx.beginPath();
    ctx.moveTo(padL, zeroY);
    ctx.lineTo(w - padR, zeroY);
    ctx.strokeStyle = '#C7D0C7';
    ctx.lineWidth = 1;
    ctx.stroke();
  }
}

function drawBarChart(canvas, labels, series, opts) {
  var setup = prepCanvas(canvas);
  if (!setup) return;
  var ctx = setup.ctx, w = setup.w, h = setup.h;
  opts = opts || {};
  var padL = 46, padR = 14, padT = 16, padB = 26;
  var plotW = w - padL - padR, plotH = h - padT - padB;

  var allValues = [];
  series.forEach(function (s) { allValues = allValues.concat(s.values); });
  var maxAbs = niceMax(Math.max.apply(null, allValues.map(Math.abs).concat([1]))
  var allowNegative = allValues.some(function (v) { return v < 0; });
  var min = allowNegative ? -maxAbs : 0;
  var max = maxAbs;

  ctx.strokeStyle = '#E8ECE8';
  ctx.fillStyle = '#98A69E';
  ctx.font = '10.5px Manrope, sans-serif';
  ctx.textBaseline = 'middle';
  var ySteps = 4;
  for (var i = 0; i <= ySteps; i++) {
    var val = min + ((max - min) / ySteps) * i;
    var y = padT + plotH - ((val - min) / (max - min)) * plotH;
    ctx.beginPath();
    ctx.moveTo(padL, y);
    ctx.lineTo(w - padR, y);
    ctx.stroke();
    ctx.textAlign = 'right';
    ctx.fillText(formatCompactNumber(val), padL - 8, y);
  }

  var zeroY = padT + plotH - ((0 - min) / (max - min)) * plotH;
  var groupW = plotW / labels.length;
  var barW = Math.min(22, (groupW * 0.7) / series.length);
  var groupInnerW = barW * series.length + (series.length - 1) * 3;

  ctx.textAlign = 'center';

```

```

ctx.textBaseline = 'top';
labels.forEach(function (label, li) {
    var groupX = padL + groupW * li + (groupW - groupInnerW) / 2;
    series.forEach(function (s, si) {
        var v = s.values[li] || 0;
        var color = s.color || CHART_COLORS[si % CHART_COLORS.length];
        var barX = groupX + si * (barW + 3);
        var barY = padT + plotH - ((v - min) / (max - min)) * plotH;
        var top = Math.min(barY, zeroY);
        var height = Math.max(1, Math.abs(barY - zeroY));
        ctx.fillStyle = color;
        roundRect(ctx, barX, top, barW, height, 3);
        ctx.fill();
    });
    ctx.fillStyle = '#98A69E';
    ctx.fillText(label, padL + groupW * li + groupW / 2, h - padB + 8);
});
}

// Рисует плавную кривую через точки (сглаживание методом усреднённых контрольных
function tracePath(ctx, pts) {
    if (!pts.length) return;
    ctx.moveTo(pts[0][0], pts[0][1]);
    if (pts.length < 3) {
        for (var i = 1; i < pts.length; i++) ctx.lineTo(pts[i][0], pts[i][1]);
        return;
    }
    for (var j = 0; j < pts.length - 1; j++) {
        var p0 = pts[j] === 0 ? 0 : j - 1;
        var p1 = pts[j];
        var p2 = pts[j + 1];
        var p3 = pts[j + 2 < pts.length ? j + 2 : j + 1];
        var cp1x = p1[0] + (p2[0] - p0[0]) / 6;
        var cp1y = p1[1] + (p2[1] - p0[1]) / 6;
        var cp2x = p2[0] - (p3[0] - p1[0]) / 6;
        var cp2y = p2[1] - (p3[1] - p1[1]) / 6;
        ctx.bezierCurveTo(cp1x, cp1y, cp2x, cp2y, p2[0], p2[1]);
    }
}

function roundRect(ctx, x, y, w, h, r) {
    if (h < r * 2) r = h / 2;
    if (w < r * 2) r = w / 2;
    r = Math.max(0, r);
    ctx.beginPath();
    ctx.moveTo(x + r, y);
    ctx.arcTo(x + w, y, x + w, y + h, r);

```

```

    ctx.arcTo(x + w, y + h, x, y + h, r);
    ctx.arcTo(x, y + h, x, y, r);
    ctx.arcTo(x, y, x + w, y, r);
    ctx.closePath();
}

function drawDonutChart(canvas, items) {
    var setup = prepCanvas(canvas);
    if (!setup) return;
    var ctx = setup.ctx, w = setup.w, h = setup.h;
    var total = items.reduce(function (acc, it) { return acc + it.value; }, 0);
    if (total <= 0) return;

    var cx = w * 0.32, cy = h / 2, radius = Math.min(cx, cy, h / 2) - 8, thickness;
    var start = -Math.PI / 2;

    items.forEach(function (it, i) {
        var angle = (it.value / total) * Math.PI * 2;
        ctx.beginPath();
        ctx.arc(cx, cy, radius, start, start + angle);
        ctx.strokeStyle = it.color || CHART_COLORS[i % CHART_COLORS.length];
        ctx.lineWidth = thickness;
        ctx.lineCap = 'butt';
        ctx.stroke();
        start += angle;
    });

    // легенда
    var legendX = w * 0.58, legendY = Math.max(14, cy - items.length * 10);
    ctx.font = '11.5px Manrope, sans-serif';
    ctx.textBaseline = 'middle';
    items.slice(0, 6).forEach(function (it, i) {
        var y = legendY + i * 20;
        ctx.fillStyle = it.color || CHART_COLORS[i % CHART_COLORS.length];
        roundRect(ctx, legendX, y - 5, 10, 10, 3);
        ctx.fill();
        ctx.fillStyle = '#3E5148';
        ctx.textAlign = 'left';
        var pct = Math.round((it.value / total) * 100);
        var label = it.label.length > 14 ? it.label.slice(0, 13) + '...' : it.label;
        ctx.fillText(label + ' · ' + pct + '%', legendX + 16, y);
    });
}

function formatCompactNumber(v) {
    var abs = Math.abs(v);
    if (abs >= 1000000) return (v / 1000000).toFixed(1).replace('.0', '') + 'M';

```

```

    if (abs >= 1000) return (v / 1000).toFixed(0) + 'K';
    return String(Math.round(v));
}

/* --- агрегация данных по месяцам --- */

function lastMonthKeys(n, refDate) {
    var keys = [];
    var now = refDate || new Date();
    for (var i = n - 1; i >= 0; i--) {
        var d = new Date(now.getFullYear(), now.getMonth() - i, 1);
        keys.push(d.getFullYear() + '-' + String(d.getMonth() + 1).padStart(2, '0'))
    }
    return keys;
}

function dateKey(d) {
    return d.getFullYear() + '-' + String(d.getMonth() + 1).padStart(2, '0') + '-'
}

function lastDayKeys(n) {
    var keys = [];
    var now = new Date();
    for (var i = n - 1; i >= 0; i--) {
        keys.push(dateKey(new Date(now.getFullYear(), now.getMonth(), now.getDate())
    }
    return keys;
}

function shortDayLabel(key) {
    var parts = key.split('-').map(Number);
    var d = new Date(parts[0], parts[1] - 1, parts[2]);
    return d.toLocaleDateString('ru-RU', { day: '2-digit', month: '2-digit' });
}

function sumByDay(items, keys, dateField, valueField, filterFn) {
    var map = {};
    items.forEach(function (it) {
        if (filterFn && !filterFn(it)) return;
        var key = it[dateField] || '';
        map[key] = (map[key] || 0) + (Number(it[valueField]) || 0);
    });
    return keys.map(function (k) { return map[k] || 0; });
}

// Баланс на момент до начала выбранного окна (для корректной кумулятивной крив
function balanceBefore(windowStartKey) {

```



```

    var cutoff = windowStartKey.length === 7 ? windowStartKey + '-01' : windowSta
    var priorIncome = sum(state.transactions.filter(function (t) { return t.type
    var priorExpense = sum(state.transactions.filter(function (t) { return t.type
    var priorSidejobs = sum(state.sidejobs.filter(function (j) { return j.status
    return priorIncome - priorExpense + priorSidejobs;
}

function shortMonthLabel(key) {
    var parts = key.split('-');
    var d = new Date(parseInt(parts[0], 10), parseInt(parts[1], 10) - 1, 1);
    return d.toLocaleDateString('ru-RU', { month: 'short' }).replace('.', '');
}

function sumByMonth(items, keys, dateField, valueField, filterFn) {
    var map = {};
    items.forEach(function (it) {
        if (filterFn && !filterFn(it)) return;
        var key = (it[dateField] || '').slice(0, 7);
        map[key] = (map[key] || 0) + (Number(it[valueField]) || 0);
    });
    return keys.map(function (k) { return map[k] || 0; });
}

function getAnalyticsReferenceDate() {
    var el = document.getElementById('analyticsMonth');
    var val = el && el.value;
    if (val && /^\\d{4}-\\d{2}$/.test(val)) {
        var parts = val.split('-');
        return new Date(parseInt(parts[0], 10), parseInt(parts[1], 10) - 1, 1);
    }
    return new Date();
}

function getAnalyticsFilteredTransactions() {
    var period = (document.getElementById('analyticsPeriod') || {}).value || 'mon
    var now = getAnalyticsReferenceDate();
    return state.transactions.filter(function (t) {
        var d = new Date(t.date);
        if (isNaN(d.getTime())) return false;
        if (period === 'month') return isSameMonth(t.date, now);
        if (period === 'quarter') { var diffMonths = (now.getFullYear() - d.getFull
        if (period === 'year') return d.getFullYear() === now.getFullYear();
        return true; // all
    });
}

function groupSumByField(items, field, valueField) {

```

```

var map = {};
items.forEach(function (it) {
    var key = it[field] || 'Дпырое';
    map[key] = (map[key] || 0) + (Number(it[valueField]) || 0);
});
return Object.keys(map)
    .map(function (k) { return { label: k, value: map[k] }; })
    .sort(function (a, b) { return b.value - a.value; });
}

function renderChartOrEmpty(containerId, canvasSelector, hasData, drawFn) {
    var container = document.getElementById(containerId);
    if (!container) return;
    var canvas = container.querySelector(canvasSelector || 'canvas');
    setChartEmpty(containerId, !hasData);
    if (hasData && canvas) drawFn(canvas);
}

function renderBalanceMiniChart() {
    var canvas = document.getElementById('balanceMiniChart');
    if (!canvas) return;
    var keys = lastMonthKeys(6);
    var windowStart = keys[0];
    var income = sumByMonth(state.transactions, keys, 'date', 'amount', function
    var expense = sumByMonth(state.transactions, keys, 'date', 'amount', function
    var sidejobsPaid = sumByMonth(state.sidejobs, keys, 'date', 'amount', functio
    var base = balanceBefore(windowStart);
    var cumulative = [];
    var running = base;
    for (var i = 0; i < keys.length; i++) {
        running += income[i] - expense[i] + sidejobsPaid[i];
        cumulative.push(running);
    }
    // Если данных нет (все значения одинаковы, обычно 0) — рисуем ровную линию,
    // а не искусственный наклон: drawSparkline сам корректно центрирует плоскую
    drawSparkline(canvas, cumulative, '#FFDD2D');
}

function renderIncomeMiniChart() {
    var canvas = document.getElementById('incomeMiniChart');
    if (!canvas) return;
    var keys = lastMonthKeys(6);
    var income = sumByMonth(state.transactions, keys, 'date', 'amount', function
    drawSparkline(canvas, income, '#00B67A');
}

function renderExpenseMiniChart() {

```

```

var canvas = document.getElementById('expenseMiniChart');
if (!canvas) return;
var keys = lastMonthKeys(6);
var expense = sumByMonth(state.transactions, keys, 'date', 'amount', function
drawSparkline(canvas, expense, '#FF4D4D');
}

var balanceHistoryPeriod = 'month';

function renderBalanceHistoryChart() {
  var hasTx = state.transactions.length > 0 || state.sidejobs.length > 0;
  renderChartOrEmpty('chartBalanceHistory', '#chartBalanceHistoryCanvas', hasTx
    var keys, labels, granularity;
    if (balanceHistoryPeriod === 'week') {
      keys = lastDayKeys(7);
      labels = keys.map(shortDayLabel);
      granularity = 'day';
    } else if (balanceHistoryPeriod === 'year') {
      keys = lastMonthKeys(12);
      labels = keys.map(shortMonthLabel);
      granularity = 'month';
    } else {
      keys = lastDayKeys(30);
      labels = keys.map(shortDayLabel);
      granularity = 'day';
    }
  }

  var income, expense, sidejobsPaid;
  if (granularity === 'day') {
    income = sumByDay(state.transactions, keys, 'date', 'amount', function (t
    expense = sumByDay(state.transactions, keys, 'date', 'amount', function (
    sidejobsPaid = sumByDay(state.sidejobs, keys, 'date', 'amount', function
  } else {
    income = sumByMonth(state.transactions, keys, 'date', 'amount', function
    expense = sumByMonth(state.transactions, keys, 'date', 'amount', function
    sidejobsPaid = sumByMonth(state.sidejobs, keys, 'date', 'amount', functio
  }

  var running = balanceBefore(keys[0]);
  // Показываем только каждую N-ю подпись на плотных периодах, чтобы подписи
  var labelStep = labels.length > 14 ? Math.ceil(labels.length / 8) : 1;
  var thinnedLabels = labels.map(function (l, i) { return (i % labelStep ===
  var cumulative = keys.map(function (k, i) { running += income[i] - expense[
  drawLineChart(canvas, thinnedLabels, [{ values: cumulative, color: '#2A8A67
  });
}

```

```

function initBalancePeriodSwitch() {
  var wrap = document.getElementById('balancePeriodSwitch');
  if (!wrap) return;
  wrap.querySelectorAll('.period-btn').forEach(function (btn) {
    btn.addEventListener('click', function () {
      if (btn.classList.contains('active')) return;
      wrap.querySelectorAll('.period-btn').forEach(function (b) { b.classList.r
      btn.classList.add('active');
      balanceHistoryPeriod = btn.getAttribute('data-period');
      renderBalanceHistoryChart();
      hapticFeedback('success');
    });
  });
}

/* ----- */
/* Подробные графики в модальных ("Денег всего" / "Остаток от дохода") */
/* ----- */

var detailBalancePeriod = 'month';

function renderBalanceDetailChart() {
  var hasTx = state.transactions.length > 0 || state.sidejobs.length > 0;
  renderChartOrEmpty('chartDetailBalanceContainer', '#chartDetailBalanceCanvas'
  var keys, labels, granularity;
  if (detailBalancePeriod === 'week') {
    keys = lastDayKeys(7);
    labels = keys.map(shortDayLabel);
    granularity = 'day';
  } else if (detailBalancePeriod === 'year') {
    keys = lastMonthKeys(12);
    labels = keys.map(shortMonthLabel);
    granularity = 'month';
  } else {
    keys = lastDayKeys(30);
    labels = keys.map(shortDayLabel);
    granularity = 'day';
  }

  var income, expense, sidejobsPaid;
  if (granularity === 'day') {
    income = sumByDay(state.transactions, keys, 'date', 'amount', function (t
    expense = sumByDay(state.transactions, keys, 'date', 'amount', function (
    sidejobsPaid = sumByDay(state.sidejobs, keys, 'date', 'amount', function
  } else {
    income = sumByMonth(state.transactions, keys, 'date', 'amount', function
    expense = sumByMonth(state.transactions, keys, 'date', 'amount', function

```

```

        sidejobsPaid = sumByMonth(state.sidejobs, keys, 'date', 'amount', function (date, amount) {
            return amount;
        });

        var running = balanceBefore(keys[0]);
        var labelStep = labels.length > 14 ? Math.ceil(labels.length / 8) : 1;
        var thinnedLabels = labels.map(function (l, i) { return (i % labelStep === 0 ? l : null); });
        var cumulative = keys.map(function (k, i) { running += income[i] - expense[i]; });
        drawLineChart(canvas, thinnedLabels, [{ values: cumulative, color: '#FFDD2D' }]);
    }

function renderBalanceDetailInsight() {
    var titleEl = document.getElementById('chartDetailBalanceInsightTitle');
    var pillEl = document.getElementById('chartDetailBalanceInsightPill');
    if (!titleEl || !pillEl) return;
    var hasTx = state.transactions.length > 0 || state.sidejobs.length > 0;
    pillEl.className = 'chart-detail__insight-pill';
    if (!hasTx) {
        titleEl.textContent = 'Пользуйтесь картой активнее';
        pillEl.textContent = 'Мало данных';
        return;
    }
    var now = new Date();
    var monthTx = state.transactions.filter(function (t) { return isSameMonth(t.date, now); });
    var monthIncome = sum(monthTx.filter(function (t) { return t.type === 'income'; }));
    var monthExpense = sum(monthTx.filter(function (t) { return t.type === 'expense'; }));
    if (monthIncome - monthExpense >= 0) {
        titleEl.textContent = 'Баланс растёт — вы тратите меньше, чем зарабатываете';
        pillEl.textContent = 'Хорошо';
        pillEl.classList.add('chart-detail__insight-pill--good');
    } else {
        titleEl.textContent = 'Баланс снижается — расходы превышают доходы';
        pillEl.textContent = 'Можно лучше';
        pillEl.classList.add('chart-detail__insight-pill--bad');
    }
}

function openBalanceDetailModal() {
    openModal('modalBalanceDetail');
    try {
        var now = new Date();
        var monthTx = state.transactions.filter(function (t) { return isSameMonth(t.date, now); });
        var allIncome = sum(state.transactions.filter(function (t) { return t.type === 'income'; }));
        var allExpense = sum(state.transactions.filter(function (t) { return t.type === 'expense'; }));
        var paidSidejobsEarned = sum(state.sidejobs.filter(function (j) { return j.date < now; }));
        var totalBalance = (allIncome - allExpense) + paidSidejobsEarned;
        setText('chartDetailBalanceValue', formatMoney(totalBalance));
    }
}

```

```

        renderBalanceDetailChart();
        renderBalanceDetailInsight();
    } catch (e) {
        console.error('openBalanceDetailModal:', e);
    }
}

function initBalanceDetailPeriodSwitch() {
    var wrap = document.getElementById('detailBalancePeriodSwitch');
    if (!wrap) return;
    wrap.querySelectorAll('.period-btn').forEach(function (btn) {
        btn.addEventListener('click', function () {
            if (btn.classList.contains('active')) return;
            wrap.querySelectorAll('.period-btn').forEach(function (b) { b.classList.r
            btn.classList.add('active');
            detailBalancePeriod = btn.getAttribute('data-period');
            renderBalanceDetailChart();
            hapticFeedback('success');
        });
    });
}

function renderCashflowDetailChart() {
    var keys = lastMonthKeys(6);
    var monthLabels = keys.map(shortMonthLabel);
    var incomeByMonth = sumByMonth(state.transactions, keys, 'date', 'amount', fu
    var expenseByMonth = sumByMonth(state.transactions, keys, 'date', 'amount', f
    var hasAnyTx = incomeByMonth.some(Boolean) || expenseByMonth.some(Boolean);
    renderChartOrEmpty('chartDetailCashflowContainer', '#chartDetailCashflowCanva
        drawBarChart(canvas, monthLabels, [
            { values: incomeByMonth, color: '#00B67A' },
            { values: expenseByMonth, color: '#FF4D4D' }
        ]);
}

function openCashflowDetailModal() {
    openModal('modalCashflowDetail');
    try {
        var now = new Date();
        var monthTx = state.transactions.filter(function (t) { return isSameMonth(t
        var monthIncome = sum(monthTx.filter(function (t) { return t.type === 'inco
        var monthExpense = sum(monthTx.filter(function (t) { return t.type === 'exp
        var net = monthIncome - monthExpense;

        setText('chartDetailCashflowValue', (net >= 0 ? '' : '-') + formatMoney(Mat
        setText('chartDetailExpenseValue', formatMoney(monthExpense));
    }
}

```

```

    setText('chartDetailIncomeValue', formatMoney(monthIncome));

    var titleEl = document.getElementById('chartDetailCashflowInsightTitle');
    var pillEl = document.getElementById('chartDetailCashflowInsightPill');
    if (titleEl && pillEl) {
        pillEl.className = 'chart-detail__insight-pill';
        if (!monthIncome && !monthExpense) {
            titleEl.textContent = 'Внесите операции, чтобы увидеть рекомендации';
            pillEl.textContent = 'Мало данных';
        } else if (monthExpense > monthIncome) {
            titleEl.textContent = 'Траты значительно больше доходов';
            pillEl.textContent = 'Можно лучше';
            pillEl.classList.add('chart-detail__insight-pill--bad');
        } else {
            titleEl.textContent = 'Доходы покрывают расходы — отличный результат';
            pillEl.textContent = 'Отлично';
            pillEl.classList.add('chart-detail__insight-pill--good');
        }
    }
}

renderCashflowDetailChart();
} catch (e) {
    console.error('openCashflowDetailModal:', e);
}
}

function initChartDetailTriggers() {
    document.addEventListener('click', function (e) {
        var el = e.target.closest ? e.target.closest('[data-open-chart]') : null;
        if (!el) return;
        try {
            var type = el.getAttribute('data-open-chart');
            if (type === 'balance') openBalanceDetailModal();
            else if (type === 'cashflow') openCashflowDetailModal();
            hapticFeedback('success');
        } catch (err) {
            console.error('chart trigger click:', err);
        }
    });
}

function renderAnalyticsCharts() {
    var refDate = getAnalyticsReferenceDate();
    var period = (document.getElementById('analyticsPeriod') || {}).value || 'mon
    var monthsSpan = period === 'quarter' ? 3 : period === 'year' ? 12 : period =
    var keys = lastMonthKeys(monthsSpan, refDate);
    var monthLabels = keys.map(shortMonthLabel);

```

```

var incomeByMonth = sumByMonth(state.transactions, keys, 'date', 'amount', function (v, i) { return v; });
var expenseByMonth = sumByMonth(state.transactions, keys, 'date', 'amount', function (v, i) { return v; });
var netByMonth = incomeByMonth.map(function (v, i) { return v - expenseByMonth[i]; });
var hasAnyTx = state.transactions.length > 0;

renderChartOrEmpty('chartIncomeDynamics', 'canvas', incomeByMonth.some(Boolea
  drawLineChart(c, monthLabels, [{ values: incomeByMonth, color: '#2A8A67', f
}));
renderChartOrEmpty('chartExpenseDynamics', 'canvas', expenseByMonth.some(Bool
  drawLineChart(c, monthLabels, [{ values: expenseByMonth, color: '#A6402C',
}));
renderChartOrEmpty('chartBalanceChange', 'canvas', hasAnyTx, function (c) {
  drawLineChart(c, monthLabels, [{ values: netByMonth, color: '#3E7CB1', fill
}));
renderChartOrEmpty('chartCashFlow', 'canvas', hasAnyTx, function (c) {
  drawBarChart(c, monthLabels, [
    { values: incomeByMonth, color: '#2A8A67' },
    { values: expenseByMonth, color: '#A6402C' }
  ]);
});

var periodTx = getAnalyticsFilteredTransactions();
var expenseCats = groupSumByField(periodTx.filter(function (t) { return t.type === 'expense'; }), 'category', 'amount');
var incomeCats = groupSumByField(periodTx.filter(function (t) { return t.type === 'income'; }), 'category', 'amount');

renderChartOrEmpty('chartExpenseByCategory', 'canvas', expenseCats.length > 0
  drawDonutChart(c, expenseCats.map(function (it, i) { return { label: it.label, value: it.amount; } }));
renderChartOrEmpty('chartIncomeDistribution', 'canvas', incomeCats.length > 0
  drawDonutChart(c, incomeCats.map(function (it, i) { return { label: it.label, value: it.amount; } }));

var expenseCountByMonth = keys.map(function (k) { return state.transactions.filter(function (t) { return t.type === 'expense' & t.date === k; }).length; });
var avgCheckByMonth = expenseByMonth.map(function (v, i) { return expenseCountByMonth[i] / v; });
renderChartOrEmpty('chartAvgCheck', 'canvas', avgCheckByMonth.some(Boolea),
  drawBarChart(c, monthLabels, [{ values: avgCheckByMonth, color: '#C0A053' }
}));

var incomeCountByMonth = keys.map(function (k) { return state.transactions.filter(function (t) { return t.type === 'income' & t.date === k; }).length; });
var avgIncomeByMonth = incomeByMonth.map(function (v, i) { return incomeCountByMonth[i] / v; });
renderChartOrEmpty('chartAvgIncomeExpense', 'canvas', avgIncomeByMonth.some(Boolea),
  drawBarChart(c, monthLabels, [
    { values: avgIncomeByMonth, color: '#2A8A67' },
    { values: avgCheckByMonth, color: '#A6402C' }
  ]);
});

```



```

var pillowByMonth = keys.map(function (k, i) { return incomeByMonth[i] > 0 ?
renderChartOrEmpty('chartPillowDynamics', 'canvas', incomeByMonth.some(Boolea
    drawLineChart(c, monthLabels, [{ values: pillowByMonth, color: '#8C6D26', f
});

var sidejobsByMonth = sumByMonth(state.sidejobs, keys, 'date', 'amount');
renderChartOrEmpty('chartSidejobsProfit', 'canvas', sidejobsByMonth.some(Bool
    drawBarChart(c, monthLabels, [{ values: sidejobsByMonth, color: '#7A5CC0' }
});

renderChartOrEmpty('chartMonthComparison', 'canvas', hasAnyTx, function (c) {
    drawBarChart(c, monthLabels, [
        { values: incomeByMonth, color: '#2A8A67' },
        { values: expenseByMonth, color: '#A6402C' }
    ]);
});

var yearKeys = lastYearKeys(3);
var incomeByYear = yearKeys.map(function (y) { return sum(state.transactions.
var expenseByYear = yearKeys.map(function (y) { return sum(state.transactions
renderChartOrEmpty('chartYearComparison', 'canvas', incomeByYear.some(Boolean
    drawBarChart(c, yearKeys, [
        { values: incomeByYear, color: '#2A8A67' },
        { values: expenseByYear, color: '#A6402C' }
    ]);
});

renderChartOrEmpty('chartBalanceForecast', 'canvas', netByMonth.some(Boolean)
    var trend = linearForecast(netByMonth, 3);
    var extendedLabels = monthLabels.concat(['+1 мес', '+2 мес', '+3 мес']);
    var lastActual = netByMonth[netByMonth.length - 1];
    // История: реальные значения, далее держим последнее значение (для непрерыв
    var historySeries = netByMonth.concat([lastActual, lastActual, lastActual])
    // Прогноз: null-заполнитель (0) для истории кроме последней точки, затем п
    var forecastSeries = netByMonth.map(function () { return null; });
    forecastSeries[forecastSeries.length - 1] = lastActual;
    forecastSeries = forecastSeries.concat(trend);
    drawLineChart(c, extendedLabels, [
        { values: historySeries, color: '#3E7CB1', fill: true },
        { values: forecastSeries.map(function (v) { return v === null ? lastActua
    ]);
});
}

function lastYearKeys(n) {
    var year = new Date().getFullYear();
    var keys = [];

```

```

    for (var i = n - 1; i >= 0; i--) keys.push(String(year - i));
    return keys;
}

function linearForecast(values, steps) {
    var n = values.length;
    var xs = values.map(function (v, i) { return i; });
    var meanX = xs.reduce(function (a, b) { return a + b; }, 0) / n;
    var meanY = values.reduce(function (a, b) { return a + b; }, 0) / n;
    var num = 0, den = 0;
    for (var i = 0; i < n; i++) { num += (xs[i] - meanX) * (values[i] - meanY); }
    var slope = den !== 0 ? num / den : 0;
    var intercept = meanY - slope * meanX;
    var out = [];
    for (var s = 1; s <= steps; s++) out.push(intercept + slope * (n - 1 + s));
    return out;
}

function monthOptionLabel(y, m) {
    var d = new Date(y, m, 1);
    var label = d.toLocaleDateString('ru-RU', { month: 'long', year: 'numeric' });
    return label.charAt(0).toUpperCase() + label.slice(1);
}

function populateAnalyticsMonthSelect() {
    var select = document.getElementById('analyticsMonth');
    if (!select) return;
    var now = new Date();
    var options = [];
    for (var i = 0; i < 24; i++) {
        var d = new Date(now.getFullYear(), now.getMonth() - i, 1);
        var value = d.getFullYear() + '-' + String(d.getMonth() + 1).padStart(2, '0');
        options.push('<option value="' + value + '">' + monthOptionLabel(d.getFullYear(), d.getMonth()));
    }
    select.innerHTML = options.join('');
}

function initAnalyticsFilters() {
    populateAnalyticsMonthSelect();
    var periodEl = document.getElementById('analyticsPeriod');
    if (periodEl) periodEl.addEventListener('change', renderAnalyticsCharts);
    var monthEl = document.getElementById('analyticsMonth');
    if (monthEl) monthEl.addEventListener('change', renderAnalyticsCharts);
}

/* ----- */
/* Открытие модальных кнопок */

```

```

/* ----- */

function initModalTriggers() {
    document.querySelectorAll('[data-close-modal]').forEach(function (btn) {
        btn.addEventListener('click', closeAllModals);
    });
    var overlay = document.getElementById('modalOverlay');
    if (overlay) {
        overlay.addEventListener('click', function (e) {
            if (e.target === overlay) closeAllModals();
        });
    }

    bind('addTransactionBtn', 'click', function () {
        document.getElementById('transactionDate').value = todayISO();
        openModal('modalTransaction');
    });
    bind('openAddTransactionModal', 'click', function () {
        document.getElementById('transactionDate').value = todayISO();
        openModal('modalTransaction');
    });
    bind('bottomAddBtn', 'click', function () {
        document.getElementById('transactionDate').value = todayISO();
        openModal('modalTransaction');
    });
    bind('openAddSidejobModal', 'click', function () {
        document.getElementById('sidejobDate').value = todayISO();
        openModal('modalSidejob');
    });
    bind('manageCategoriesBtn', 'click', function () { renderCategoriesList(); op

    bind('modalConfirmActionBtn', 'click', function () {
        if (typeof confirmCallback === 'function') confirmCallback();
        confirmCallback = null;
        closeAllModals();
    });
}

function bind(id, event, handler) {
    var el = document.getElementById(id);
    if (el) el.addEventListener(event, handler);
}

/* ----- */
/* Вспомогательные */
/* ----- */

```

```

function sum(arr, field) {
    return arr.reduce(function (acc, item) { return acc + (Number(item[field]) || 0); }, 0);
}

function setText(id, text) {
    var el = document.getElementById(id);
    if (el) el.textContent = text;
}

function escapeHtml(str) {
    if (str === undefined || str === null) return '';
    return String(str)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;');
}

function hapticFeedback(type) {
    try {
        var hapticsEnabled = document.getElementById('settingsTelegramHaptics');
        if (hapticsEnabled && hapticsEnabled.checked && window.Telegram && window.Telegram.WebApp.HapticFeedback.notificationOccurred(type);
    } catch (e) { /* тихо игнорируем, если Telegram SDK недоступен */ }
}

function initTelegram() {
    try {
        if (window.Telegram && window.Telegram.WebApp) {
            var tg = window.Telegram.WebApp;
            tg.ready();
            tg.expand();
            var user = tg.initDataUnsafe && tg.initDataUnsafe.user;
            if (user) {
                var nameEl = document.getElementById('userName');
                var avatarEl = document.getElementById('userAvatar');
                if (nameEl) nameEl.textContent = [user.first_name, user.last_name].filter(Boolean).join(' ');
                if (avatarEl) avatarEl.textContent = (user.first_name || 'U').charAt(0);
            }

            // tg.expand() и разворот клавиатуры/смена темы меняют реальный
            // размер вьюпорта уже ПОСЛЕ того, как графики отрисовались.
            // Обычный window 'resize' не всегда срабатывает внутри
            // Telegram WebView, поэтому подписываемся на нативное событие SDK.
            if (typeof tg.onEvent === 'function') {
                tg.onEvent('viewportChanged', function () {

```

```

        if (window.requestAnimationFrame) {
            requestAnimationFrame(function () { renderVisibleCharts(); });
        } else {
            renderVisibleCharts();
        }
    });
}

}

} catch (e) { /* приложение открыто вне Telegram — это нормально */ }
}

/* ----- */
/* Инициализация */
/* ----- */

function renderAll() {
    populateTransactionCategorySelect(getActiveTransactionType());
    populateFilterCategorySelect();
    renderDashboard();
    renderTransactionsTable();
    renderSidejobs();
    renderAnalyticsCharts();
}

// Перерисовывает ВСЕ графики на странице.
// Нужен отдельно от renderAll(), потому что canvas, лежащий внутри
// display:none (неактивная вкладка/страница или закрытая модалка),
// получает нулевую ширину/высоту в getBoundingClientRect() и рисуется
// "схлопнутым". Поэтому графики нужно перерисовывать каждый раз,
// когда их контейнер становится видимым, а не только один раз при
// старте приложения.
function renderVisibleCharts() {
    renderBalanceMiniChart();
    renderIncomeMiniChart();
    renderExpenseMiniChart();
    renderBalanceHistoryChart();
    renderAnalyticsCharts();

    var balanceModal = document.getElementById('modalBalanceDetail');
    if (balanceModal && !balanceModal.hidden) renderBalanceDetailChart();

    var cashflowModal = document.getElementById('modalCashflowDetail');
    if (cashflowModal && !cashflowModal.hidden) renderCashflowDetailChart();
}

function init() {
    initTelegram();
}

```

```

initSidebar();
initNavigation();
initModalTriggers();
initCategories();
initTransactionForm();
initTransactionFilters();
initSidejobForm();
initSidejobFilters();
initSettings();
initAnalyticsFilters();
initBalancePeriodSwitch();
initBalanceDetailPeriodSwitch();
initChartDetailTriggers();

document.getElementById('transactionDate') && (document.getElementById('trans
document.getElementById('sidejobDate') && (document.getElementById('sidejobDa

renderAll();
window.addEventListener('resize', debounce(renderVisibleCharts, 200));
}

if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', init);
} else {
  init();
}
})();

```